ConsumerAuthorization

System Description

Abstract

This document describes the ConsumerAuthorization core system of the Eclipse Arrowhead 5.2.2 framework, as implemented in Java Spring in `ah5-core-java-spring/consumer-authorization`, including the local extensions of this workspace.

Contents

1 Overview **3**
1.1 Significant Prior Art . . . 4
1.2 How This System Is Meant to Be Used . . . 4
1.3 System functionalities and properties . . . 4
1.4 Important Delimitations . . . 5

2 Services **6**
2.1 Produced services . . . 6
2.2 Consumed services . . . 6

3 Security **7**
3.1 Security model . . . 7

4 References **7**

5 Revision History **9**
5.1 Amendments . . . 9
5.2 Quality Assurance . . . 9

1 Overview

This document describes the ConsumerAuthorization system. ConsumerAuthorization decides which consumer systems may use which services of which providers. It stores authorization policies at two levels (management and provider), evaluates authorization questions for orchestrators and providers, and issues and verifies authorization tokens.



Figure 1: SysML-style block view of ConsumerAuthorization with its produced and consumed services.

The rest of this document is organized as follows. In Section 1.1, we reference major prior art capabilities of the system. In Section 1.2, we describe the intended usage of the system. In Section 1.3, we describe fundamental properties provided by the system. In Section 1.4, we describe delimitations of capabilities of the system. In Section 2, we describe the abstract service functions consumed or produced by the system. In Section 3, we describe the security capabilities of the system.

1.1 Significant Prior Art

ConsumerAuthorization continues the Arrowhead 4.x Authorization system (intra-cloud rules and token generation). Arrowhead 5 adds policy types (`ALL`, `WHITELIST`, `BLACKLIST`, `SYS_METADATA`), per-operation scoped policies, a management level that overrides provider policies, and several token variants (time-limited, usage-limited, self-contained, JSON Web Tokens).

1.2 How This System Is Meant to Be Used

Providers publish the policies of their own services with `authorization.grant`; operators and configuration scripts set management-level policies with `authorizationManagement`. DynamicServiceOrchestration asks `check-policies` for each candidate and obtains tokens with `generate-tokens` when a provider interface requires them. Providers verify tokens with `authorizationToken.verify` or locally with the public key.

1.3 System functionalities and properties

1.3.1 Functional properties of the system

- Store and evaluate authorization policies: management level first, provider level if none applies; no policy means denied.
- Policy types `ALL`, `WHITELIST`, `BLACKLIST` and `SYS_METADATA` (consumer metadata looked up with `systemDiscovery.lookup`); default and per-operation policies.
- Generate and verify tokens: time-limited and usage-limited simple tokens, Base64 self-contained tokens (optionally encrypted with the provider's AES key), RSA-SHA256/512 JSON Web Tokens.
- Management of tokens and provider encryption keys; periodic removal of old tokens.

1.3.2 Configuration of system properties

The system is configured with Spring properties (`application.properties`, overridden by environment variables in the container deployment). The properties that shape its behaviour are:

Property	Default	Meaning
`server.port`	8445	HTTPS port.
`authentication.policy`	declared	Test cloud: `outsourced` with the Authentication secret key.
`arrowhead.security.profile`	secure	`development` selects the isolated development exception; never set in a deployment.
`management.policy` / `management.whitelist`	authorization	Access to the management services.
`token.max.age`	60	Minutes after which tokens are deleted.
`token.time.limit`	60	Default lifetime of time-limited tokens and JWTs in seconds.
`simple.token.byte.size` / `simple.token.usage.limit`	32 / 5	Size and default uses of simple tokens.
`unbounded.token.generation.whitelist`	DynamicServiceOrchestration, TranslationManager	Systems that may generate tokens without authorization check.
`secret.cryptographer.key`	(per-cloud secret)	16-byte key encrypting tokens and keys in the database.
`enable.blacklist.filter` / `force.blacklist.filter`	false / true	Blacklist check of requesters.

Property	Default	Meaning
`normalization.mode`	extended	Name normalization.
`mqtt.api.enabled`	false	Offer the MQTT interfaces.
`sysmonitor.enabled`	false	Publish native sysMonitor observations (test cloud: true).
`arrowhead.ssl.crl-file`	(unset)	CRL checked for every client certificate (local).

Secrets (database, keystore and identity passwords, secret keys) have no usable defaults; in the test clouds they are per-cloud secrets mounted as files.

1.3.3 Data stored by the system

Policy headers per level (management, provider) with their policies, generated tokens (time-limited, usage-limited, self-contained) with a token header, provider encryption keys, cryptographic auxiliary data, and the log entries (database `ah_consumer_authorization`). Tokens and keys are encrypted with `secret.cryptographer.key` before storage.

1.3.4 Non functional properties

Policy evaluation is a database lookup per question; `SYS_METADATA` policies add a ServiceRegistry lookup. The RSA key pair for JSON Web Tokens is taken from the system's keystore.

1.3.5 Stateful or stateless

Stateful: policies, tokens and keys are persistent.

1.4 Important Delimitations

ConsumerAuthorization does not enforce access itself; enforcement relies on the orchestrator returning only authorized providers and on providers checking tokens. A provider can only manage its own provider-level policies. Event types are a supported target type but are only used by event-handling systems.

2 Services

This section describes the produced and consumed services. Each produced service is specified by an SD document and one IDD document per interface.

2.1 Produced services

Service	Version	Interfaces	SD	IDD
`authorization`	1.0.0	HTTP(S), MQTT(S)	[1]	[2], [3]
`authorizationToken`	1.0.0	HTTP(S), MQTT(S)	[4]	[5], [6]
`authorizationManagement`	1.0.0	HTTP(S), MQTT(S)	[7]	[8], [9]
`authorizationToken Management`	1.0.0	HTTP(S), MQTT(S)	[10]	[11], [12]
`generalManagement`	1.0.0	HTTP(S), MQTT(S)	[13]	[14], [15]
`sysMonitor`	1.0.0	AMQP(S)	[16]	[17]

The HTTP interfaces are always offered, the MQTT interfaces with `mqtt.api.enabled=true` and the AMQP `sysMonitor` interface with `sysmonitor.enabled=true`. Upstream 5.2.2 also answers `GET <base>/monitor/echo` on some systems; this endpoint is not registered as a service.

2.2 Consumed services

Provider	Service	Operations	Use	Condition
ServiceRegistry	serviceDiscovery, systemDiscovery	register, revoke, lookup	Registration; consumer metadata for `SYS_METADATA` policies.	always
Authentication	identity	identity-login, identity-verify	Own token; verification of requester tokens.	`authentication.policy=outsourced`
Blacklist	blacklistDiscovery	check	Blacklist filter.	`enable.blacklist.filter=true`
AMQP broker (SysMonitor)	sysMonitor (publisher)	echo, power, status	Own observations.	`sysmonitor.enabled=true`

Except for the bootstrap services (`http.collector.bootstrap.services`), which are looked up in the ServiceRegistry, the system reaches other systems' services through DynamicServiceOrchestration (`http.collector.mode=ORCHESTRATION`, results cached for `http.collector.orchestration.ttl` = 300 s); every such call needs a ConsumerAuthorization grant.

3 Security

The default is the Arrowhead secure mode with HTTPS and X.509 certificates following the Arrowhead certificate profile (`<System>.<Cloud>.<Organization>.arrowhead.eu`). Certificates not issued under the cloud CA are refused by the TLS handshake. The Arrowhead-test-cloud-java deployment runs the secure mode.

The packaged `application.properties` is a development sample that disables TLS and declares identities. The system refuses to start with it, or with any other effective configuration that sets `server.ssl.enabled=false`, `authentication.policy=declared`, `service.authorization.enabled=false` or, while monitoring is on, `sysmonitor.ssl.enabled=false`. The check runs before the system accepts a connection. Only an explicit `arrowhead.security.profile=development` lets such a system start, for an isolated test, and it then logs a non-conformance warning.

3.1 Security model

- **Protocols:** HTTPS (TLS 1.3 and 1.2) with mandatory client certificates (`server.ssl.client-auth=need`); MQTT over TLS; AMQP over TLS for `sysMonitor`.
- **Data protection:** transport encryption by TLS; the database connection uses TLS with server verification in the test cloud. Secrets are kept out of the container environment (Compose file secrets).
- **System authentication:** by the authentication policy: `certificate` (client certificate CN), `outsourced` (identity token verified with Authentication), `internal` (Authentication only) or `declared` (insecure). In this workspace an identity token is accepted only together with the client certificate of the system it was issued to (token-to-certificate binding), and every client certificate is checked against the cloud CA's CRL when `arrowhead.ssl.crl-file` is set (fail closed).
- **Authorization of produced services:** management services are protected by the management filter (`management.policy`: `SYSOP_ONLY`, `WHITELIST` or `AUTHORIZATION` via ConsumerAuthorization); an optional blacklist filter refuses blacklisted requesters. Sysop identities bypass the management filter.
- Management-level policies override provider-level ones, so operators can restrict what providers publish. In the test clouds the grants every inter-system call needs are set by `configure-orchestration-dependency-policies.sh` and the other policy scripts; a missing grant makes orchestration return no result.

For the Arrowhead certificate profile see github.com/eclipse-arrowhead/documentation.

4 References

[1] J. Delsing, "authorization – Service Description (ConsumerAuthorization)," Eclipse Arrowhead, SD 5.2.2, 2026, consumerAuthorization-v5.2.2-authorization-SD.pdf.

[2] ——, "authorization HTTP/TLS/JSON – Interface Design Description (ConsumerAuthorization)," Eclipse Arrowhead, IDD 5.2.2, 2026, consumerAuthorization-v5.2.2-authorization-HTTP-IDD.pdf.

[3] ——, "authorization MQTT/TLS/JSON – Interface Design Description (ConsumerAuthorization)," Eclipse Arrowhead, IDD 5.2.2, 2026, consumerAuthorization-v5.2.2-authorization-MQTT-IDD.pdf.

[4] ——, "authorizationToken – Service Description (ConsumerAuthorization)," Eclipse Arrowhead, SD 5.2.2, 2026, consumerAuthorization-v5.2.2-authorizationToken-SD.pdf.

[5] ——, "authorizationToken HTTP/TLS/JSON – Interface Design Description (ConsumerAuthorization)," Eclipse Arrowhead, IDD 5.2.2, 2026, consumerAuthorization-v5.2.2-authorizationToken-HTTP-IDD.pdf.

[6] ——, "authorizationToken MQTT/TLS/JSON – Interface Design Description (ConsumerAuthorization)," Eclipse Arrowhead, IDD 5.2.2, 2026, consumerAuthorization-v5.2.2-authorizationToken-MQTT-IDD.pdf.

[7] ——, “authorizationManagement – Service Description (ConsumerAuthorization),” Eclipse Arrowhead, SD 5.2.2, 2026, consumerAuthorization-v5.2.2-authorizationManagement-SD.pdf.

[8] ——, “authorizationManagement HTTP/TLS/JSON – Interface Design Description (ConsumerAuthorization),” Eclipse Arrowhead, IDD 5.2.2, 2026, consumerAuthorization-v5.2.2-authorizationManagement-HTTP-IDD.pdf.

[9] ——, “authorizationManagement MQTT/TLS/JSON – Interface Design Description (ConsumerAuthorization),” Eclipse Arrowhead, IDD 5.2.2, 2026, consumerAuthorization-v5.2.2-authorizationManagement-MQTT-IDD.pdf.

[10] ——, “authorizationTokenManagement – Service Description (ConsumerAuthorization),” Eclipse Arrowhead, SD 5.2.2, 2026, consumerAuthorization-v5.2.2-authorizationTokenManagement-SD.pdf.

[11] ——, “authorizationTokenManagement HTTP/TLS/JSON – Interface Design Description (ConsumerAuthorization),” Eclipse Arrowhead, IDD 5.2.2, 2026, consumerAuthorization-v5.2.2-authorizationTokenManagement-HTTP-IDD.pdf.

[12] ——, “authorizationTokenManagement MQTT/TLS/JSON – Interface Design Description (ConsumerAuthorization),” Eclipse Arrowhead, IDD 5.2.2, 2026, consumerAuthorization-v5.2.2-authorizationTokenManagement-MQTT-IDD.pdf.

[13] ——, “generalManagement – Service Description (ConsumerAuthorization),” Eclipse Arrowhead, SD 5.2.2, 2026, consumerAuthorization-v5.2.2-generalManagement-SD.pdf.

[14] ——, “generalManagement HTTP/TLS/JSON – Interface Design Description (ConsumerAuthorization),” Eclipse Arrowhead, IDD 5.2.2, 2026, consumerAuthorization-v5.2.2-generalManagement-HTTP-IDD.pdf.

[15] ——, “generalManagement MQTT/TLS/JSON – Interface Design Description (ConsumerAuthorization),” Eclipse Arrowhead, IDD 5.2.2, 2026, consumerAuthorization-v5.2.2-generalManagement-MQTT-IDD.pdf.

[16] ——, “sysMonitor – Service Description (ConsumerAuthorization),” Eclipse Arrowhead, SD 5.2.2, 2026, consumerAuthorization-v5.2.2-sysMonitor-SD.pdf.

[17] ——, “sysMonitor AMQP/TLS/JSON – Interface Design Description (ConsumerAuthorization),” Eclipse Arrowhead, IDD 5.2.2, 2026, consumerAuthorization-v5.2.2-sysMonitor-AMQP-IDD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-11-07	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	